

We won't get into how you can use promises in your code since it's out of scope, however if you often write asynchronous code, they are definitely worth checking out.

Generators

Imagine a function that keeps running even after it has returned a value. In fact it can keep returning values as long as needed. On the other hand, you can call such a function, and what you get is an object that can keep returning the next value when you ask for it. This is a concept that is popular in Python, and is now part of ES6.

Let's look at a simple generator that returns consecutive odd numbers starting with 1:

```
function* oddNumbers() {
  let num = 1;
  while(true) {
    yield num;
    num += 2;
  }
}
```

You'll notice two odd things about the above function. The first is that there is `*` following function. This is not a misprint, it is the syntax to define a generator function. The second oddity is the presence of the `yield` keyword instead of the usual `return`. Where `return` returns a value and ends the execution of the function that returned the value, `yield` continues executing even after the value has been generated.

In the above case you could use the generator as follows:

```
let oddNums = oddNumbers();
oddNums.next(); // returns {value: 1, done: false}
oddNums.next(); // returns {value: 3, done: false}
oddNums.next(); // returns {value: 5, done: false}
```

It is also possible to create an iterator using a generator which would allow you to use it in expressions such as `for(num of oddNumbers) [...]`.

How to Use ES6 Today

We wouldn't tease you with the good stuff that is coming with future versions of JavaScript if there wasn't a way to start using it already.

While a lot of these features are still not fully supported across the browser spectrum, most engines have already implemented

over half of these features. Microsoft is for once leading the charge in implementing these new features and if you are looking to play with these new features you should check them out on their new Edge browser.



You can find out how to configure Babel for your workflow at babeljs.io



With the release of Node 4.0 a lot of ES6 features such as proxies, promises, arrow functions etc. are available in Node.

Fact is though, you will need to support old JavaScript for a long time, even if all browsers reach 100% support for ES6 over the next few months. It obviously makes little sense to code everything twice, once in ES6 and once in ES5, that would defeat the point of the improvements in ES6.

If you want to start using ES6 already but want to still support older browsers, you should check out a project called Babel that compiles ES6 JavaScript to ES5 JavaScript, allowing you to take advantage of the new syntax today while still targeting older browser users. Once you feel ES6 has achieved the level of mass acceptance you want, you can simply use the ES6 code directly instead of passing it through Babel. >

*addons



*Microsoft's early source code out!

>>Microsoft's OpenSSH, a Windows port code for developers is published and welcomes all you coders to get your hands on it.
<http://dgit.in/Nv15bq>



*Bad Programming habits we love!

>>As coders we all have pesky habits which we know aren't good, but as usual we end up breaking the rules. And love it!
<http://dgit.in/Nv15br>



*OpenBSD turns 20

>>OpenBSD, the unix-like free to use operating system turns 20. On its anniversary, a new 5.8 version with a long list of improvements was released.
<http://dgit.in/Nv15bs>